Scaling Software Engineering Agent Training Across Data and Model Size

Qingyao Sun (qs234), Weitao Su(ws535)

1. Problem

Recent work such as SWE-Gym, SWE-smith, and SWE-Master demonstrates that post-training can improve software engineering agent performance. However, these systems differ in training data, model size, training methodology, and agent framework, making it difficult to isolate the factors driving performance improvements.

We aim to study how training-data source and model size affect agent performance and computational efficiency by fine-tuning Qwen2.5-Coder 1.5B, 3B, and 7B using data derived from SWE-Gym, SWE-smith, and SWE-Master.

Research Questions:

- How do training-data source and model size affect task resolution performance under standardized supervised fine-tuning (SFT) or Direct Preference Optimization (DPO)?
- What performance improvements are achieved relative to training and inference costs?
- How do smaller fine-tuned models compare with larger models and published state-of-the-art (SOTA) systems?

Our goal is to identify the performance–compute trade-offs between smaller fine-tuned agents and larger software engineering models through controlled experiments.

2. Challenges

Fair Comparisons: Existing systems use different models, frameworks, training methods, and inference budgets. We standardize the model family, scaffold, training procedure, and evaluation settings. Published SOTA results will serve as reference comparisons.

Dataset Quality and Overlap: The three datasets differ in size and trajectory quality, while SWE-Master incorporates data from overlapping sources. We will standardize formats, remove identifiable overlaps, check for benchmark contamination, and construct approximately equal-token subsets.

Training Methodology: SOTA systems may combine SFT, DPO, reinforcement learning (RL), and test-time scaling. We focus on SFT/DPO to isolate data and model-size effects, using published results to contextualize other methodologies.

Compute Cost: Nine fine-tuning runs and twelve evaluation conditions are expensive. We use LoRA and approximately 100 SWE-bench Verified tasks, prioritizing core experiments over optional analyses.

3. Approach

We will build a common agent pipeline that can inspect repositories, modify code, run tests, incorporate feedback, and submit patches.

Data Preparation: Standardize SWE-Gym, SWE-smith, and SWE-Master-derived trajectories, remove overlaps, and construct equal-token training subsets. Record dataset statistics, including task counts, trajectory lengths, and token distributions.

Training: Fine-tune Qwen2.5-Coder 1.5B/3B/7B on each dataset using consistent LoRA configurations and training-token budgets. Record training loss, GPU hours, memory usage, and throughput.

Model Scaling: Evaluate nine fine-tuned models against three corresponding base-model baselines to measure performance improvements across datasets and model sizes.

SOTA Comparison: Compare results with published larger-model systems, documenting differences in training and evaluation methodologies. If compute permits, evaluate an unmodified Qwen2.5-Coder-32B model using our scaffold.

4. Evaluation

We will evaluate all models on the same approximately 100-task subset of SWE-bench Verified using identical agent and inference settings.

- **Performance:** Task resolution rate (Pass@1), improvement over base models, and scaling across datasets and model sizes.
- **Training Efficiency:** GPU hours, memory usage, throughput, and performance gains relative to training cost.
- **Inference Efficiency:** Token consumption, tool calls, agent steps, runtime, and timeout rates.
- **SOTA Comparison:** Performance relative to published larger-model results, with differences in evaluation conditions explicitly documented.
- **Failure Analysis:** Categorize representative failures involving incorrect localization, patch generation, test feedback, and execution limits.

Success Criteria: Complete all twelve experimental conditions, obtain reproducible resolution measurements, and quantify how training-data source and model size affect performance and computational costs. We will report uncertainty and distinguish controlled results from published comparisons. Outperforming SOTA is not required.

5. Timeline & Task Assignments

Period	Activities
Sep 14–27	Set up SWE-bench, Qwen models, and agent scaffold.
Sep 28–Oct 11	Prepare SWE-Gym data and validate LoRA training.
Oct 12–25	Process remaining datasets and construct matched-token subsets.
Oct 26–Nov 8	Fine-tune 1.5B and 3B models.
Nov 9–22	Fine-tune 7B models and begin evaluation.
Nov 23–Dec 6	Complete evaluation, analyze results, and finalize report and presentation.

Roles:
Weitao Su: Dataset preparation, fine-tuning, and training efficiency.
Qingyao Sun: Agent scaffold, evaluation, performance analysis, and visualization.
All: Experiments, final report, and presentation.

6. Deliverables

- A reproducible agent training and evaluation pipeline with standardized training datasets.
- Nine fine-tuned Qwen models and three base-model baselines.
- SWE-bench Verified results across all twelve conditions.
- Performance-scaling and computational-efficiency comparisons with larger SOTA systems.
- Failure analysis and reproducible scripts, configurations, and experiment logs.
- A final report and presentation documenting methodology, findings, and limitations.

Final Outcome: A controlled empirical study of how training data and model scale influence software engineering agent performance, identifying trade-offs between fine-tuning smaller models and deploying larger models.